

上海交通大学 试 卷 (__ 卷)

(学年 第 学期)

班级号 _____ 学号 _____ 姓名 _____

课程名称 _____ 计算机系统结构 _____ 成绩 _____

一、单项选择题：

1. 某数采用 IEEE 754单精度浮点数格式表示为 C6600000 H，则该数的值是 ()

- A. -1.75×2^{13} B. -1.75×2^{12} C. -0.75×2^{13} D. -0.75×2^{12}

2. 以下一段MIPS指令序列：

```
addi    $t1, $zero, 30    #R[$t1]←30
lw      $t2, 16($a0)      #R[$t2]←M[R[$a0]+16]
add     $v0, $t1, $t2      #R[$v0] ←R[t1]+R[t2]
```

以上指令序列中，假定采用“取指、译码/取数、执行、访存、写回”这5段流水线方式，不采用“转发（前向通路, forwarding, bypassing）”技术；时钟周期的长度足够让写寄存器发生在一个周期的上半段、读寄存器发生在一个周期的后半段；需要在第3条指令（add）前加入 () 条空操作（nop）指令才能使这段程序不发生数据冒险。

- A.0 B.1 C.2 D.3

3. 某计算机存储器按字节编址，采用小端方式放数据。假定编译器规定int 型和 short型 长度分别为 32 位和 16 位，并且数据按边界对齐存储。语言程序段如下：

Struct	若 record 变量的首地址是 0xC008, record.c 的地址是
{ char a;	()
int b;	A. 0xC00D
short c;	B. 0xC00E
} record;	C. 0xC00F
	D. 0xC010

4. 下列代码的输出是？ ()

```
int x = 0x15213F10 >> 4;
char y = (char) x;
unsigned char z = (unsigned char) x;
printf("%d, %u", y, z); // 将 y 和 z 分别以整数、无符号整数形式输出
```

- (A) -241, 15 (B) -15, 241 (C) -241, 241 (D) -15, 15

我承诺，我将严格遵守考试纪律。

题号										
得分										
批阅人(流水阅)										

5. 如果一台能支持硬件同时多线程（ Simultaneous Multithreading (SMT) ）的机器，它的资源是有限的，以下哪种硬件资源会直接限制机器可以支持的总线程数？（ ）

- A. 功能单元的数目
- B. 物理寄存器的个数
- C. 数据高速缓存的容量大小
- D. 数据高速缓存的关联度

6. 给定一个处理器有六个阶段的流水线，流水段分别为: F D I X0 X1 W

读寄存器发生在 I 段（issue 段），分支是否转移要在X1段才确定, 如果流水线是单发射的，每个时钟周期只发射一条指令，那么当执行条件转移指令、转移预测错误时，需要废除多少条已经进入流水线的指令？（假设进入流水线的第一条指令是条件转移指令）（ ）

- A. 2条
- B. 3条
- C. 4条
- D. 5条

7. 已知数组元素在内存中存放是行主序，即按行存放。下面的代码中，循环采用哪种顺序可以获得最好的局部性（locality）？（ ）

```
// int a[X][Y][Z] is declared earlier,
int i, j, k, sum = 0;
for (i = 0; i < Y; i++)
    for (j = 0; j < Z; j++)
        for (k = 0; k < X; k++)
            sum += a[i][j][k];
```

- A. i 在最外层, j 在中间层, k 在最里层 (如上所示).
- B. j在最外层, k在中间层, i在最里层
- C. k在最外层, i在中间层, j 在最里层
- D. 循环顺序和局部性无关

8. 以下对SIMD的描述，哪一个是正确的？（ ）

- A. SIMD 是一种指令级并行（instruction-level parallelism）
- B. SIMD 能提升控制流语句比较多的程序的性能（例如：有较多的转移指令、if语句的程序）
- C. Intel的 SIMD intrinsic 指令使用了字长比较长的寄存器，这样可以一次同时对多个数据进行同一个操作。
- D. SIMD是将多条指令同时执行，处理多个数据流，例如：多线程的程序就是SIMD技术

9. 哪种技术可以在执行时提升以下代码的处理速度？ ()

```
LD R1 0(R2)      # cache miss
ADD R2 R1 R1      # R2 是目标寄存器
LD R1 0(R3)      # cache hit
LD R3 0(R4)      # cache hit
ADD R3 R1 R3      # R3 是目标寄存器
ADD R1 R2 R3      # R1 是目标寄存器
```

-
- A. 带寄存器换名的乱序执行 (out of order with register renaming)
 B. 转移预测(branch prediction)
 C. 多发射 (multiple issue)
 D. 同时多线程 (simultaneous multi-thread)

10. 如果处理器采用五阶段顺序流水线，并支持精确中断。对于发生异常的单条指令，中断被称为“精确”当且仅当满足：

- (1) 所有该指令之前的指令都已经提交其状态；
 (2) 所有后续指令（包括中断的指令）没有改变任何机器的状态。

下图中，不同的指令在执行的不同阶段发生了各种异常 (exception)，是哪条指令的异常最先被处理？ ()



- A. 发生 Data TLB fault 异常的指令
 B. 发生 Bad instruction 异常的指令
 C. 发生 Inst TLB fault 异常的指令
 D. 发生 Overflow 异常的指令

11. 定义了以下变量：

```
int x = random();
int y = random();
int z = random();
/* Convert to other forms */
unsigned ux = (unsigned) x;
unsigned uy = (unsigned) y;
```

从以下表达式中，选出一项不永为真 (true) 的表达式： ()

- A. $((x+y) < 4) + y - x == 17*y + 15*x$
 B. $\sim x + \sim y + 1 == \sim(x+y)$
 C. $ux - uy == -(y-x)$
 D. $(x < y) == (-x > -y)$

二、简答题/ 计算题

1. 假定机器M的字长为32位，用补码表示带符号整数。下表第一列给出了在机器M上执行的C语言程序中的关系表达式，请参照已有的表栏内容完成表中后三栏内容的填写。

关系表达式	运算类型	结果	说明
0 == 0U	无符号整数	1	00...0B = 00...0B
-1 < 0	有符号整数	1	11...1B (-1) < 00...0B (0)
-1 < 0U			
2147483647 > -2147483647 - 1			
2147483647U > -2147483647 - 1			
2147483647 > (int) 2147483648U			
-1 > -2			
(unsigned) -1 > -2			

2. 简单介绍写直达和写回法高速缓存的不同之处 (the difference between a write-through and a write-back cache.)

3. 采用IEEE 754单精度浮点数格式计算下列表达式的值。
0.75+(- 65.25)

4. 用2-3句话，简答：为什么现代处理器会使用多级高速缓存（例如：三级高速缓存 Level 1, Level 2, and sometimes Level 3 cache）。

5. 简述：如何在超标量处理器中支持线程级的并行性？即“同时多线程”（ Simultaneous Multithreading (SMT) ）处理器的工作原理。

6. 下表给出了Cache的参数,请填写表格中空白的区域.其中: m是内存物理地址的位数, C是Cache的大小 (以字节为单位), B是高速缓存块的大小 (block size, 以字节为单位), E是高速缓存的组内关联度(set associativity), S是高速缓存中组的数目 (number of sets), t是tag所占的位数, s是组索引位的长度 (或者称为组号的长度), b是表示块内偏移量所需要的位数。

m	C	B	E	S	T	s	b
32	8192	32	4				

7. 假定系统采用2级页表（ two-level page translation） 页面大小为4KB（4K-byte pages） 每一个页表项的长度为4个字节（4-byte page table entries）. 那么一个32位的虚拟地址，分为几段？ 每段表示什么含义？为什么是这样分段？

8. 假设同一套指令集用不同的方法设计了两种机器M1和M2。机器M1的时钟周期为0.8ns， 机器M2的时钟周期为1.2ns。某个程序P在机器M1上运行时的CPI为4， 在M2上的CPI为2。对于程序P来说， 哪台机器的执行速度更快？ 快多少？

9. 你准备使用卷积神经网络（ convolutional network）对100,000 幅图像进行分类。假设你可以使用的计算机有32个核， 你的程序并行运行32 个线程来进行图像分类。该项目99%的工作量可以被并行化分配到这32个线程中处理， 整个程序可以获得的加速比是多少？

三、综合分析题

1. 指令流水线

考虑以下代码，它的功能是将一组向量中的每一个浮点数与一个常量相乘（The code we consider scales a vector of floating-point numbers by a constant.）

```
I1:  loop: LD.D F0, 0(R1) // 从内存将一个元素读入浮点寄存器 F0
I2:  MUL.D F0, F2, F0      // 乘以常量
I3:  ST.D F0, 0(R1)       // 将运算结果存入内存
I4:  DDI R1, R1, 8        // 修改数组下标
I5:  BNE R1, R2, loop     // 数组未结束则跳转
```

(1) 找出上述指令之间的数据相关性

(2) 接下来，我们考虑一个单发射、具有前向通路的按序流水线(the fully-bypassed, single-issue, in-order pipeline) 如下图所示， 前向通路未画出。 Load 指令读内存的结果要在 X3 流水段的结尾才能获得，浮点乘法的结果要在 X4 流水段的结尾才能获得。Store 指令要写入内存的数据要在 X2 流水段时准备好即可，在 X2 流水段前不需要该数据。 假设分支指令的转移预测总是正确的 (没有因为 branch 指令导致的流水线停顿: there are no bubbles due to branches.)

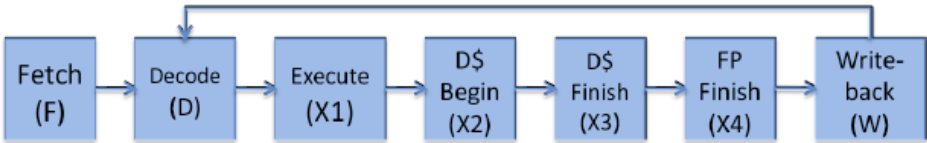


Figure 1. In-order pipeline. Full bypassing is not shown.

下面表格是两次循环的指令执行时空图。填写表格中给出的指令在流水线中的执行情况。在适合的单元格中，给出在不同周期时，指令所处于的流水段的名称（用 F 、D、X1、X2、X3、X4 和 W 表示各流水段）。

Cycle	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
Inst																									
I1:iteration1	F	D	X1	X2	X3	X4	W																		
I2:iteration1		F	D	D	D	X1	X2	X3	X4	W															
I3:iteration1																									
I4:iteration1																									
I5:iteration1																									
I1:iteration2																									
I2:iteration2																									
I3:iteration2																									
I4:iteration2																									
I5:iteration2																									

如果表格中的书写空间不够，你可以在下面空白处，重新画时空图。

2. 分析下列代码:

```
1. int x[2][256];
2. int i;
3. int sum = 0;
4. for ( i=0; i< 256; i++) {
5.     sum +=  x[0][i] * x[1][i];
6. }
```

- `sizeof ( int )= 4.`
- 数组 `x` 的内存地址从 `0x0` 开始, 在内存中的存储是行主序 (`stored in row-major order`: 按行存放).
- 在下面的每一种情况, `cache` 都初始化为空。
- 所有的内存访问都是对数组 `x` 的元素, 其他变量都存储在寄存器中。

给定以下假设, 估计在每种不同情况时, 高速缓存的失效率 (`miss rates`) .

- 情况 1: 假定高速缓存大小为 `1024 bytes`, 直接映射 (`direct-mapped`), 高速缓存块大小为 `32` 字节 (`32-byte cache blocks`). 求高速缓存的失效率 (`miss rates`) ?
- 情况 2: 如果情况 1 其他参数不变, 将 `cache` 大小增加到 `2048 bytes`, 求高速缓存的失效率 (`miss rates`) ?
- 情况 3: 现在假定高速缓存大小为 `1024 bytes`, 使用 2 路组关联映射 (`two-way set associative`)、`LRU` 替换算法, 高速缓存块大小为 `32` 字节 (`32-byte cache blocks`). 求高速缓存的失效率 (`miss rates`) ?

3. 考虑一个由 3 个处理器构成的基于总线的共享存储器系统。该共享存储器被分为 x,y,z,w 四块。每个处理器有一个高速缓存，且在任何给定时间只能安装一块。每一块可能处于两个状态之一：有效 (V) 或无效 (I)。假定高速缓存开始时已被清空，且存储器的内容如下：

存储器块	x	y	z	w
内容	20	30	50	20

考虑遵循以下给定顺序的存储器访问事件序列：

- 1) P1: Read(x) 2) P2: Read(x) 3) P3: Read(x) 4) P1: x=x+15
 5) P1: Read(y) 6) P2: x=x+15 7) P3: Read(x) 8) P1: y=y+10

高速缓存的采用的协议为：写回法(write back)和写无效(write invalidate)协议。说明在上述的每一次操作后高速缓存和存储器的内容以及高速缓存块的状态。

事件	P1 的高速缓存	P2 的高速缓存	P3 的高速缓存	主存			
				x	y	z	w
初始	(I) 清空	(I) 清空	(I) 清空	20	30	50	20
P1: Read (x)	(V) 20	(I) 清空	(I) 清空	20	30	50	20
P2: Read (x)	(V) 20	(V) 20	(I) 清空	20	30	50	20
P3: Read (x)	(V) 20	(V) 20	(V) 20	20	30	50	20
P1: x=x+15							
P1: Read (y)							
P2: x=15							
P3: Read (x)							
P1: y=y+10							

4. 分析以下每一段代码，它是否和对应的串行代码有相同的功能，并且比串行代码快？如果是错误的或者并没有比串行代码加速，请说明原因。

A.

```
//将 arr 第 i 个元素的值设置为 I; Set element i of arr to i
#pragma omp parallel {
for (int i = 0; i < n; i++) arr[i] = i;
}
```

B.

```
// 将 arr 数组元素设置为斐波那契数列: Set arr to be an array of Fibonacci numbers.
arr[0] = 0;
arr[1] = 1;
#pragma omp parallel for
for (int i = 2; i < n; i++)
    arr[i] = arr[i-1] + arr[i - 2];
```

C.

```
// 将 arr 所有元素设置为 0; Set all elements in arr to 0;
int i;
#pragma omp parallel for
for (i = 0; i < n; i++) arr[i] = 0;
```

D.

```
// 将 arr 数组的每一个元素减去其数组下标值
//Decrements element i of arr. n is a multiple of omp_get_num_threads()
#pragma omp parallel {
    int threadCount = omp_get_num_threads();
    int myThread = omp_get_thread_num();
    for (int i = 0; i < n; i++) {
        if (i % threadCount == myThread) arr[i] -=i;
    }
}
```